

Exploração de Grafos em uma Rede de Pré-requisitos do NICEN

Camylle Millene Lima Bezerra¹, Tawanny TÁCyla da Silva²

Resumo: Este trabalho apresenta a implementação e a análise de algoritmos clássicos de grafos aplicados à rede de pré-requisitos de disciplinas do Núcleo Interdisciplinar de Ciências Exatas e da Natureza (NICEN). A rede foi modelada como um grafo dirigido e implementada em Python, utilizando lista de adjacência, sem o uso de bibliotecas específicas para grafos. Foram implementados os algoritmos de Busca em Largura (BFS), Busca em Profundidade (DFS) e o algoritmo de Kosaraju para identificação das componentes fortemente conexas, além de funcionalidades para obtenção de informações da rede, identificação do vértice de maior grau, verificação de ciclos e determinação das disciplinas sem pré-requisitos. Os resultados permitiram caracterizar a estrutura da rede, identificar o menor caminho entre disciplinas, construir a árvore da DFS e verificar a inexistência de ciclos. O projeto permitiu aplicar, na prática, conceitos fundamentais sobre grafos e analisar uma rede real de pré-requisitos por meio de algoritmos clássicos.

Palavras-chave: Grafos; Busca em Largura; Busca em Profundidade; Componentes Fortemente Conexas; Pré-requisitos.

1 INTRODUÇÃO

Os grafos constituem uma importante estrutura de dados utilizada para representar relações entre diferentes elementos (WEST, 2001). Essa forma de modelagem é aplicada em diversas áreas, como redes sociais, sistemas de transporte, redes de computadores, mapas, cadeias de produção e organização curricular. A principal vantagem dessa representação é facilitar a modelagem e a análise de problemas que envolvem relações entre diferentes elementos.

Neste trabalho foi utilizada uma rede de pré-requisitos entre disciplinas do Núcleo Interdisciplinar de Ciências Exatas e da Natureza (NICEN). Nessa rede, cada disciplina é representada por um vértice e cada relação de pré-requisito é representada por uma aresta direcionada. Como uma disciplina pode depender da conclusão de outra para ser cursada, as conexões possuem sentido definido, caracterizando a estrutura como um grafo dirigido.

A análise dessa rede é importante porque permite compreender como as disciplinas estão organizadas ao longo da matriz curricular. Além de identificar quais disciplinas funcionam como base para outras, também é possível verificar a existência de caminhos entre componentes da rede, detectar possíveis ciclos, analisar o grau de conectividade entre as disciplinas e identificar aquelas que podem ser cursadas sem a necessidade de conhecimentos prévios. Essas informações podem contribuir para uma melhor compreensão da estrutura curricular e do encadeamento dos conteúdos oferecidos pelo curso.

Essas análises são aplicações clássicas dos algoritmos de grafos (CORMEN et al., 2012).

Para realizar essas análises, foi desenvolvida uma aplicação em Python utilizando apenas estruturas e algoritmos implementados pelas autoras, sem o uso de bibliotecas específicas para grafos, conforme estabelecido na proposta da disciplina. O grafo foi representado por meio de uma lista de adjacência, estrutura escolhida por apresentar boa eficiência para redes esparsas, como é o caso da rede de pré-requisitos analisada.

Entre os algoritmos implementados estão a Busca em Largura (BFS), utilizada para determinar caminhos mínimos em grafos sem peso; a Busca em Profundidade (DFS), empregada para explorar a rede e registrar os tempos de descoberta e finalização dos vértices; e o algoritmo de Kosaraju, utilizado para identificar as componentes fortemente conexas (CORMEN et al., 2012).

Também foram implementadas funcionalidades complementares, como o cálculo do número de vértices, do número de arestas e do grau médio da rede, a identificação do vértice de maior grau, a verificação da existência de ciclos e a determinação das disciplinas que não possuem pré-requisitos.

Dessa forma, o principal objetivo deste trabalho é aplicar os conceitos estudados na disciplina de Algoritmos e Estruturas de Dados II em um conjunto de dados real, analisando a rede de pré-requisitos por meio de algoritmos clássicos de grafos e interpretando os resultados obtidos de acordo com o contexto da estrutura curricular representada.

2 IMPLEMENTAÇÃO

Nesta seção são apresentadas as principais decisões adotadas durante o desenvolvimento do projeto, bem como a organização do código e as estruturas de dados utilizadas para representar e analisar a rede de pré-requisitos. Cada algoritmo foi implementado em uma classe independente, facilitando a organização do projeto e a reutilização do código.

2.1 REPRESENTAÇÃO DA REDE

A primeira decisão do projeto foi definir como a rede de pré-requisitos seria representada. Como cada disciplina pode ser pré-requisito de outra, optou-se por utilizar um grafo dirigido, em que os vértices representam as disciplinas e as arestas representam as relações de dependência entre elas.

Os dados da rede foram obtidos a partir dos arquivos D2_prereqs-nicen.csv, contendo as relações entre as disciplinas, e D2_prereqs-nicen-nodes.csv, com a identificação dos vértices. Durante a execução, essas informações são lidas e utilizadas para construir o grafo.

Para armazenar a estrutura da rede foi escolhida uma lista de adjacência, implementada com um dicionário em Python. Essa representação foi escolhida porque a rede possui poucas conexões em relação ao número de disciplinas, consumindo menos memória do que uma matriz de adjacência e facilitando a execução dos algoritmos de busca.

Além da lista de adjacência, também foram armazenados um conjunto com todos os vértices e um contador de arestas, utilizados posteriormente para calcular informações como o número de vértices, o número de arestas e o grau médio da rede.

2.2 ESTRUTURA DA CLASSE GRAFO

A classe Grafo foi desenvolvida para concentrar a construção da rede e as operações básicas utilizadas pelos demais algoritmos. Nela estão reunidos os métodos responsáveis pela leitura do arquivo CSV, inserção das arestas e armazenamento das conexões entre as disciplinas.

Para representar o grafo foram utilizadas três estruturas principais: um dicionário para a lista de adjacência, um conjunto contendo todos os vértices e um contador para registrar o número total de arestas. Essas estruturas foram suficientes para armazenar toda a rede e servir de base para as demais funcionalidades implementadas.

A construção do grafo acontece durante a leitura do arquivo CSV. Cada linha representa uma relação de pré-requisito, que é adicionada automaticamente à

lista de adjacência. Além da construção da rede, a classe também reúne funções auxiliares utilizadas ao longo do projeto, como o cálculo do grau médio, a geração do grafo transposto, a identificação do vértice de maior grau, a verificação de ciclos e a busca por disciplinas sem pré-requisitos.

2.3 ALGORITMOS IMPLEMENTADOS

Para atender aos objetivos do projeto, foram implementados os principais algoritmos estudados na disciplina. Cada um desempenha uma função específica na análise da rede, desde a busca por caminhos até a identificação de componentes fortemente conexas.

2.3.1 IMPLEMENTAÇÃO DA BUSCA EM LARGURA (BFS)

A Busca em Largura (BFS) foi implementada em uma classe própria para manter a organização do projeto. Durante a execução, é utilizada uma fila para controlar a ordem de visita dos vértices, além de um conjunto de visitados e um dicionário que registra o predecessor de cada disciplina.

Sempre que um novo vértice é encontrado, ele é marcado como visitado, inserido na fila e tem seu predecessor armazenado. Ao final da busca, esse dicionário permite reconstruir o menor caminho entre dois vértices.

Com essa implementação foi possível responder uma das questões propostas no trabalho, identificando corretamente o menor caminho entre as disciplinas MAT001 e MAT004, além da ordem de visita produzida pela BFS.

2.3.2 IMPLEMENTAÇÃO DA BUSCA EM PROFUNDIDADE (DFS)

A Busca em Profundidade (DFS) também foi implementada em uma classe independente. Diferentemente da BFS, a exploração ocorre seguindo um caminho até o ponto em que não existam mais vértices a visitar, retornando apenas para continuar pelos demais ramos da rede.

A implementação foi feita de forma recursiva. Durante a execução são registrados os vértices visitados, a ordem da busca, os tempos de entrada e saída e o pai de cada vértice, informações utilizadas para construir a árvore da DFS.

Esses dados também serviram de base para outras funcionalidades do projeto, como a identificação das componentes fortemente conexas e a verificação da existência de ciclos.

2.3.3 IDENTIFICAÇÃO DAS COMPONENTES FORTEMENTE CONEXAS

Para identificar as componentes fortemente conexas da rede, foi utilizado o algoritmo de Kosaraju. A escolha desse algoritmo ocorreu por ser uma solução conhecida para esse tipo de problema e por aproveitar a própria DFS implementada anteriormente.

O processo acontece em duas etapas. Primeiro é realizada uma busca em profundidade no grafo original para registrar a ordem de finalização dos vértices. Em seguida, é criado o grafo transposto, invertendo o sentido das arestas, e uma nova DFS é executada seguindo essa ordem. Cada busca realizada nessa segunda etapa corresponde a uma componente fortemente conexa.

Essa funcionalidade permitiu identificar todas as componentes presentes na rede e responder uma das questões propostas no trabalho.

2.4 FUNCIONALIDADES ADICIONAIS

Além dos algoritmos clássicos de busca e da identificação das componentes fortemente conexas, foram implementadas funcionalidades complementares com o objetivo de obter informações sobre a estrutura da rede e responder às questões propostas no trabalho. Essas análises permitiram caracterizar o grafo e interpretar suas principais propriedades de forma mais completa.

2.4.1 INFORMAÇÕES GERAIS DA REDE

Inicialmente foram implementados métodos responsáveis pelo cálculo das principais características do grafo. Essas informações correspondem ao número de vértices, ao número de arestas e ao grau médio da rede.

O número de vértices é obtido diretamente a partir do conjunto que armazena todas as disciplinas presentes na rede. Como um conjunto não permite elementos repetidos, cada disciplina é contabilizada apenas uma vez, independentemente da quantidade de relações de pré-requisito das quais participe.

O número de arestas é determinado por meio da variável utilizada como contador durante a construção do grafo. Sempre que uma nova relação entre duas disciplinas é inserida, esse contador é incrementado, evitando a necessidade de percorrer novamente toda a estrutura para obter essa informação.

O grau médio é calculado dividindo-se a soma dos graus dos vértices pelo número total de vértices existentes na rede. Como o grafo analisado é

dirigido, foi considerado o número de conexões armazenadas na lista de adjacência de cada disciplina. Essa medida fornece uma visão geral da quantidade média de relações existentes entre as disciplinas da matriz curricular.

Essas informações são apresentadas logo no início da execução do programa e servem para caracterizar a rede antes da aplicação dos demais algoritmos.

2.4.2 IDENTIFICAÇÃO DO VÉRTICE DE MAIOR GRAU

Outra funcionalidade implementada foi a identificação da disciplina que apresenta o maior número de conexões na rede. Para realizar essa análise, o algoritmo percorre todas as listas de adjacência do grafo, calculando o número de vizinhos associados a cada vértice. Durante esse processo são armazenados o maior grau encontrado até o momento e a disciplina correspondente.

Ao término da verificação, o programa apresenta o vértice de maior grau juntamente com seu respectivo valor. No conjunto de dados analisado, a disciplina MAT001 apresentou o maior número de conexões, totalizando seis relações de pré-requisito.

A identificação desse vértice permite reconhecer disciplinas que desempenham papel importante na estrutura curricular, pois concentram um número maior de dependências em comparação às demais. Essas disciplinas costumam representar conteúdos fundamentais para o desenvolvimento das etapas seguintes do curso.

2.4.3 VERIFICAÇÃO DA EXISTÊNCIA DE CICLOS

Também foi implementado um procedimento para verificar se a rede de pré-requisitos apresenta ciclos. Em uma estrutura curricular, espera-se que não existam dependências circulares entre disciplinas. Caso uma disciplina dependesse, direta ou indiretamente, dela própria, seria impossível estabelecer uma sequência válida para cursá-la, comprometendo toda a organização da matriz curricular.

A verificação foi realizada utilizando a própria Busca em Profundidade. Durante a exploração da rede, além do conjunto de vértices visitados, é mantido um conjunto correspondente aos vértices que pertencem ao caminho recursivo atual da busca. Sempre que um vértice alcança outro que ainda se encontra nesse caminho, identifica-se a existência de um ciclo.

Caso nenhum retorno desse tipo seja encontrado durante toda a execução do algoritmo, conclui-se que o grafo é acíclico. Portanto, para a rede analisada,

não foram identificados ciclos, resultado compatível com a natureza de uma rede de pré-requisitos acadêmicos, cuja organização deve impedir dependências circulares entre disciplinas.

2.4.4 IDENTIFICAÇÃO DAS DISCIPLINAS SEM PRÉ - REQUISITOS

Por fim, foi implementada uma funcionalidade para identificar as disciplinas que podem ser cursadas sem aprovação prévia em outra disciplina. Para isso, todos os vértices são inicialmente considerados candidatos a não possuir pré-requisitos.

Em seguida, o algoritmo percorre as arestas do grafo e registra as disciplinas que aparecem como destino das relações de dependência, indicando que possuem pelo menos um pré-requisito. Ao final desse processo, as disciplinas que não receberam nenhuma aresta de entrada são identificadas como aquelas que não possuem pré-requisitos e podem ser cursadas desde o início da graduação.

Essa funcionalidade permitiu responder a uma das perguntas propostas para a análise da rede, identificando as disciplinas que podem ser cursadas logo no início da graduação. Além de responder ao problema proposto, essa informação fornece uma visão geral sobre a organização curricular e auxilia na compreensão da estrutura de dependências existente entre as disciplinas.

3 ANÁLISE DE COMPLEXIDADE

Depois de concluir a implementação, analisamos o custo dos principais algoritmos utilizados no projeto. Como o conjunto de dados utilizado possui apenas 36 vértices e 38 arestas, todos os algoritmos executam rapidamente. Mesmo assim, é importante entender como eles se comportariam caso a rede fosse muito maior. As complexidades apresentadas correspondem às análises clássicas descritas na literatura para esses algoritmos (CORMEN et al., 2012).

A construção do grafo acontece logo após a leitura do arquivo CSV. Durante esse processo, cada relação de pré-requisito é adicionada à lista de adjacência correspondente. Como todas as arestas são processadas apenas uma vez, essa etapa apresenta custo $O(V + E)$. O espaço utilizado também é $O(V + E)$, pois é necessário armazenar os vértices e suas respectivas conexões.

Na implementação da BFS, utilizamos uma fila para controlar a ordem de visita dos vértices. Além disso, foi criado um dicionário para registrar o predecessor de cada disciplina, permitindo reconstruir o menor caminho encontrado ao final da busca. Durante a execução, cada vértice é visitado apenas uma vez e cada aresta é explorada uma única vez. Dessa forma,

o algoritmo possui complexidade linear, representada por $O(V + E)$, utilizando $O(V)$ de memória auxiliar.

A implementação da DFS segue a mesma ideia da BFS quanto ao controle dos vértices visitados, porém utiliza chamadas recursivas em vez de uma fila. Durante a busca também registramos os tempos de entrada e saída de cada vértice, informações que posteriormente foram utilizadas para montar a árvore da DFS e auxiliar em outras análises da rede. Assim como na BFS, todos os vértices e arestas são percorridos uma única vez, fazendo com que o algoritmo apresente o mesmo custo assintótico, $O(V + E)$.

Para encontrar as componentes fortemente conexas utilizamos o algoritmo de Kosaraju, já que o grafo de pré-requisitos é dirigido. Primeiro foi executada uma DFS para obter a ordem de finalização dos vértices. Em seguida, foi criado o grafo transposto e realizada uma nova DFS seguindo essa ordem. Embora o algoritmo execute duas buscas em profundidade, cada vértice e cada aresta são processados apenas um número constante de vezes, resultando também em um custo de $O(V + E)$.

A identificação da disciplina com maior grau foi uma das implementações mais simples do projeto. Foi necessário apenas percorrer as listas de adjacência do grafo, comparando a quantidade de conexões associadas a cada disciplina. Como cada vértice é analisado apenas uma vez, essa operação apresenta complexidade $O(V)$.

A verificação da existência de ciclos também foi feita durante uma busca em profundidade. Enquanto os vértices eram explorados, foi mantido um controle daqueles que ainda estavam na pilha de recursão. Caso um vértice fosse encontrado novamente enquanto ainda permanecia nessa pilha, seria identificado um ciclo. Como a verificação acompanha a própria execução da DFS, seu custo permanece $O(V + E)$.

Também foi analisada a função responsável por identificar as disciplinas que não possuem pré-requisitos. Para isso, o algoritmo percorre todas as relações existentes na rede para verificar quais disciplinas aparecem como destino das arestas, ou seja, quais dependem de outra disciplina. Em seguida, essas informações são comparadas com o conjunto de todas as disciplinas cadastradas no grafo, permitindo identificar aquelas que não possuem nenhuma dependência. Como cada vértice e cada aresta são percorridos apenas uma vez durante esse processo, essa funcionalidade possui complexidade $O(V + E)$.

As implementações da BFS, DFS e suas análises de complexidade também seguem a abordagem clássica

apresentada por Sedgewick e Wayne (2011). Os resultados mostram que a representação por lista de adjacência foi adequada para a rede analisada. Como existem apenas 38 relações entre 36 disciplinas, a rede possui poucas conexões em relação ao número de disciplinas, o que reduz o consumo de memória e permite que os algoritmos de busca sejam executados de forma eficiente. Embora a rede utilizada seja pequena, as implementações continuam eficientes mesmo para conjuntos de dados maiores, já que todos os principais algoritmos empregados apresentam crescimento linear em função do número de vértices e arestas.

Tabela 1 – Complexidade de tempo e espaço dos algoritmos e funcionalidades implementados.

Algoritmo	Tempo	Espaço
Construção do grafo	$O(V + E)$	$O(V + E)$
BFS	$O(V + E)$	$O(V)$
DFS	$O(V + E)$	$O(V)$
Kosaraju	$O(V + E)$	$O(V + E)$
Identificação do maior grau	$O(V)$	$O(1)$
Verificação de ciclos	$O(V + E)$	$O(V)$
Disciplinas sem pré-requisitos	$O(V + E)$	$O(V)$

Fonte: Elaborada pelas autoras (2026).

A Tabela 1 resume as complexidades dos principais algoritmos implementados, permitindo comparar seus custos de tempo e espaço.

4 RESPOSTAS DAS PERGUNTAS

4.1 QUANTOS COMPONENTES FORTEMENTE CONEXOS A REDE POSSUI? O GRAFO É CONEXO?

Para responder a essa questão, foi utilizado o algoritmo de Kosaraju, responsável por identificar as componentes fortemente conexas em grafos dirigidos. Após a execução do programa, verificou-se que a rede possui 36 componentes fortemente conexas.

Esse resultado é esperado porque o grafo representa uma rede de pré-requisitos entre disciplinas. Nessa estrutura, uma disciplina pode ser pré-requisito de

outra, porém não existe um caminho de retorno para a disciplina de origem, respeitando a direção das arestas. Dessa forma, cada disciplina constitui uma componente fortemente conexa individual.

Assim, conclui-se que o grafo não é fortemente conexo, pois não é possível alcançar todos os vértices a partir de qualquer outro seguindo o sentido das arestas. Portanto, embora todas as disciplinas pertençam à mesma rede curricular, elas não são mutuamente alcançáveis quando se respeita a direção das arestas.

Saída do programa:

COMPONENTES FORTEMENTE CONEXAS

Quantidade de componentes: 36

Componente 1: COM001

Componente 2: QUI001

Componente 3: BIO001

...

Componente 34: HUM001

Componente 35: HUM002

Componente 36: FIS007

4.2 QUAL É O MENOR CAMINHO ENTRE DOIS NÓS DE SUA ESCOLHA? JUSTIFIQUE A ESCOLHA.

Para responder a essa questão, foi utilizada a Busca em Largura (BFS), algoritmo apropriado para determinar o menor caminho em grafos não ponderados.

Foram escolhidas as disciplinas MAT001 e MAT004 por fazerem parte da sequência inicial de disciplinas de Matemática da matriz curricular. Além disso, existe uma cadeia de pré-requisitos entre elas, permitindo verificar se o algoritmo identifica corretamente a sequência mínima necessária para alcançar a disciplina de destino. Após a execução da BFS, foi encontrado o seguinte caminho:

Saída do programa:

Menor caminho entre MAT001 e MAT004:

MAT001 -> MAT002 -> MAT003 -> MAT004

O resultado mostra que, para chegar à disciplina MAT004, é necessário cursar anteriormente MAT002 e MAT003, formando um caminho com três arestas. Como a Busca em Largura sempre encontra o

caminho com o menor número de arestas em grafos não ponderados, conclui-se que essa é a menor sequência de pré-requisitos entre as duas disciplinas escolhidas.

Esse resultado também demonstra que a implementação da BFS funcionou corretamente, identificando o menor percurso existente entre os dois vértices da rede.

4.3 QUAL É O NÓ DE MAIOR GRAU? O QUE ESSE NÓ REPRESENTA NO CONTEXTO REAL DA REDE?

Para responder a essa questão, foi implementada uma função responsável por identificar o vértice com maior grau na rede. O algoritmo percorre todas as listas de adjacência, comparando a quantidade de conexões de cada disciplina até encontrar aquela que apresenta o maior número de relações. Após a execução do programa, foi identificado o seguinte resultado.

Saída do programa:

Nó de maior grau:

Vértice: MAT001

Grau: 6

A análise mostrou que a disciplina MAT001 possui o maior grau da rede, totalizando seis conexões. Esse resultado indica que ela se relaciona diretamente com mais disciplinas do que qualquer outra presente na matriz curricular.

No contexto da rede de pré-requisitos, esse resultado mostra que MAT001 desempenha um papel fundamental na estrutura curricular, pois serve como base para diversas disciplinas que dependem dos conhecimentos adquiridos nela. Dessa forma, essa disciplina ocupa uma posição de destaque na organização da matriz curricular, sendo uma das primeiras etapas da formação dos estudantes.

Esse resultado era esperado, já que MAT001 é uma disciplina oferecida no início do curso e serve como pré-requisito para várias outras. Por isso, é natural que ela apresente um número maior de conexões na rede.

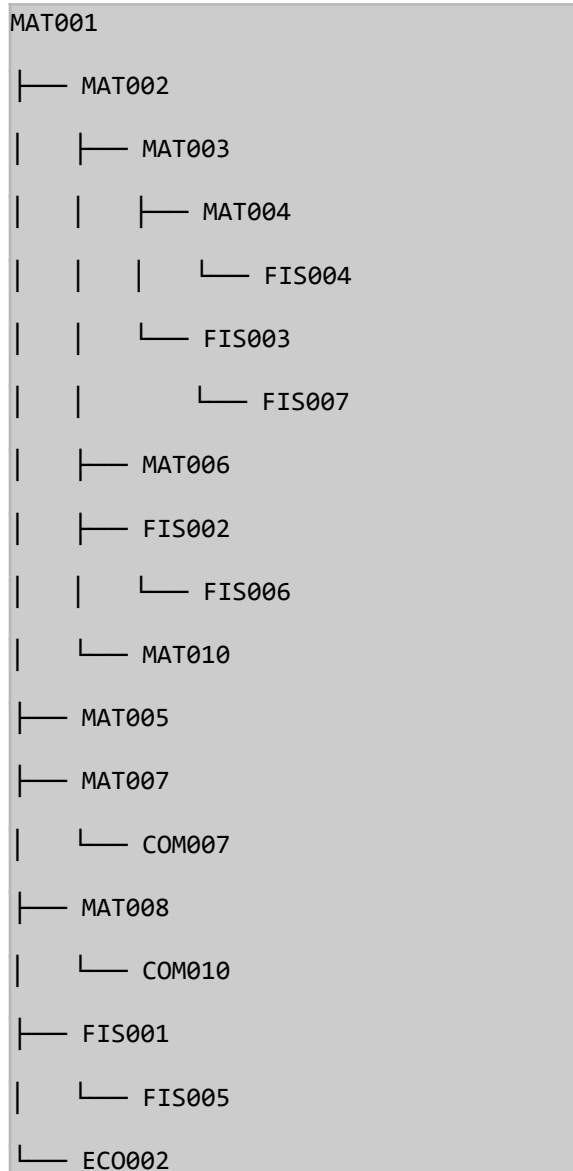
4.4 EXECUTE A DFS A PARTIR DE UM NÓ DE SUA ESCOLHA E APRESENTE A ÁRVORE DE DFS RESULTANTE. IDENTIFIQUE AS ARESTAS DE RETORNO, SE HOUVER.

A busca em profundidade foi iniciada a partir do vértice MAT001, por ser uma das disciplinas que dá origem a várias sequências de pré-requisitos dentro da rede. Durante a execução, o algoritmo registrou a

ordem em que os vértices foram visitados e armazenou o pai de cada um deles, permitindo construir a árvore da DFS.

A árvore obtida mostra como a busca percorreu a rede seguindo os caminhos disponíveis até não haver mais vértices para visitar, retornando apenas quando necessário para continuar a exploração.

Figura 1 – Árvore de busca em profundidade (DFS) iniciada no vértice MAT001.



Fonte: Elaborada pelas autoras (2026).

A Figura 1 apresenta a rede construída a partir dos dados fornecidos para o projeto. Pela visualização, é possível identificar como as disciplinas estão conectadas por meio das relações de pré-requisito, permitindo compreender melhor a organização da matriz curricular e as dependências entre elas.

Saída do programa:

Árvore da DFS:

```
MAT001: raiz da árvore
MAT002: pai = MAT001
MAT003: pai = MAT002
MAT004: pai = MAT003
FIS004: pai = MAT004
FIS003: pai = MAT003
...
FIS001: pai = MAT001
FIS005: pai = FIS001
ECO002: pai = MAT001
```

Durante a execução da DFS, não foram encontradas arestas de retorno. Esse resultado era esperado, já que a rede de pré-requisitos não apresenta dependências circulares entre as disciplinas.

4.5 O GRAFO POSSUI CICLOS? COMO ISSO FOI DETERMINADO?

Para verificar se a rede de pré-requisitos possui ciclos, foi implementada uma função baseada na Busca em Profundidade (DFS). Durante a execução do algoritmo, além de marcar os vértices já visitados, também foi mantido um conjunto contendo os vértices que ainda estavam sendo explorados. Se, durante a busca, um desses vértices fosse encontrado novamente antes do término da exploração, isso indicaria a existência de um ciclo. Após a execução do programa, foi obtido o seguinte resultado:

Saída do programa:

Verificação de ciclos:

O grafo não possui ciclos.

O resultado mostra que a rede analisada não possui ciclos, ou seja, não existem dependências circulares entre as disciplinas. Isso significa que uma disciplina nunca depende, direta ou indiretamente, dela mesma para ser cursada.

Esse comportamento é esperado em uma rede de pré-requisitos, pois a existência de ciclos impediria a definição de uma sequência válida de disciplinas ao longo do curso. Dessa forma, a verificação realizada confirma que a estrutura da rede é consistente e compatível com a organização da matriz curricular.

4.6 PERGUNTA PROPOSTA PELA EQUIPE: QUAIS DISCIPLINAS NÃO POSSUEM PRÉ-REQUISITOS?

Como sexta pergunta, foi proposta a identificação das disciplinas que não possuem pré-requisitos.

Essa pergunta é interessante porque permite identificar quais disciplinas podem ser cursadas logo no início da graduação. Além disso, ela ajuda a compreender como a matriz curricular está

organizada, destacando os pontos de entrada da rede de pré-requisitos.

Enquanto as perguntas anteriores analisavam as relações de dependência entre as disciplinas, esta última busca identificar aquelas que podem ser cursadas desde o início da graduação.

Para responder a essa questão, foi implementada uma função que percorre todas as relações de pré-requisito do grafo e identifica as disciplinas que não possuem arestas de entrada, ou seja, aquelas que não dependem da aprovação em nenhuma outra disciplina. Após a execução do programa, foi obtido o seguinte resultado:

Saída do programa:

Disciplinas sem pré-requisitos:

BIO001

COM001

ECO001

HUM001

MAT001

MAT009

QUI001

Total: 7

A análise mostrou que a rede possui sete disciplinas sem pré-requisitos. Essas disciplinas representam o ponto de partida da matriz curricular, podendo ser cursadas sem a necessidade de aprovação em outras disciplinas.

Essa informação pode auxiliar os estudantes no planejamento da graduação, pois indica quais disciplinas podem ser cursadas logo nos primeiros períodos. Além disso, mostra que a matriz curricular oferece diferentes caminhos para o início da formação, permitindo ao estudante ingressar em áreas distintas do curso desde os primeiros períodos.

5 CONCLUSÃO

O desenvolvimento deste trabalho possibilitou aplicar, na prática, os conceitos estudados na disciplina de Algoritmos e Estruturas de Dados II por meio da análise de uma rede real de pré-requisitos acadêmicos. A implementação da representação do grafo por lista de adjacência e dos algoritmos de Busca em Largura (BFS), Busca em Profundidade (DFS) e Kosaraju permitiu compreender o

funcionamento dessas técnicas e sua aplicação na resolução de problemas relacionados a grafos dirigidos.

Os resultados obtidos mostraram que a rede analisada apresenta uma estrutura compatível com uma matriz curricular organizada por relações de dependência. A análise identificou que o grafo possui 36 componentes fortemente conexas, não apresenta ciclos, tem a disciplina MAT001 como o vértice de maior grau e possui sete disciplinas que podem ser cursadas sem pré-requisitos. Além disso, foi possível determinar o menor caminho entre disciplinas e construir a árvore gerada pela Busca em Profundidade, atendendo aos objetivos propostos no trabalho.

Durante o desenvolvimento do projeto também foram aprimorados conhecimentos relacionados à leitura de arquivos CSV, manipulação de estruturas de dados em Python, organização do código em diferentes módulos e interpretação dos resultados produzidos pelos algoritmos implementados. A separação das funcionalidades em classes contribuiu para uma implementação mais organizada e facilitou a reutilização do código.

Como limitação, destaca-se que a análise foi realizada sobre uma rede relativamente pequena, composta por 36 disciplinas e 38 relações de pré-requisito. Embora os algoritmos utilizados apresentem boa eficiência para conjuntos de dados maiores, não foi possível avaliar seu desempenho em redes de maior porte. Além disso, o estudo considerou apenas as relações de pré-requisito, sem incluir informações como período recomendado, carga horária ou outros aspectos da matriz curricular.

De modo geral, os objetivos estabelecidos para o projeto foram alcançados. A implementação e a análise dos algoritmos permitiram compreender melhor a estrutura da rede de pré-requisitos e demonstraram como os conceitos de grafos podem ser aplicados na análise de problemas reais, reforçando a importância dessas técnicas tanto no estudo de algoritmos quanto em aplicações práticas.

6 USO DE FERRAMENTAS DE IA

Durante o desenvolvimento deste trabalho foi utilizada a ferramenta ChatGPT como apoio na revisão de trechos do código e na compreensão de alguns algoritmos implementados.

Um dos trechos revisados com auxílio da ferramenta foi a implementação do algoritmo de Kosaraju, utilizado para identificar as componentes fortemente conexas do grafo. O auxílio concentrou-se na organização das etapas do algoritmo, especialmente na construção do grafo transposto e na definição da

seqüência de execução das duas buscas em profundidade (DFS), garantindo que a implementação estivesse de acordo com o funcionamento esperado do algoritmo.

Além da revisão do algoritmo, a ferramenta também foi utilizada como apoio na elaboração da documentação do projeto, auxiliando na criação de um modelo para o arquivo HISTORICO_COMMITS.md, que registra a evolução do desenvolvimento do projeto por meio dos commits realizados no GitHub. A estrutura sugerida serviu apenas como referência para organizar as informações, sendo posteriormente adaptada e preenchida pelas autoras com base no histórico real do repositório e nas implementações realizadas em cada etapa do desenvolvimento.

Após as sugestões fornecidas pela ferramenta, o código foi revisado, adaptado e testado pelas autoras antes de sua inclusão na versão final do projeto.

6.1 ANÁLISE CRÍTICA DO CÓDIGO GERADO POR IA

Durante a implementação do algoritmo de Kosaraju, o ChatGPT foi utilizado como ferramenta de apoio para revisar a lógica do algoritmo e esclarecer dúvidas sobre a ordem das etapas de execução. Inicialmente, as explicações fornecidas eram bastante genéricas e não estavam totalmente adaptadas à estrutura do projeto, que utilizava uma classe própria para representar o grafo.

Por esse motivo, foi necessário adaptar as sugestões à implementação desenvolvida pela equipe, integrando o algoritmo às estruturas de dados já existentes e utilizando os métodos da classe Grafo, como a geração do grafo transposto. Após essas adaptações e testes, foi possível obter corretamente os 36 componentes fortemente conexas da rede.

7 REFERÊNCIAS

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

GOODRICH, Michael T.; TAMASSIA, Roberto; GOLDWASSER, Michael H. *Estruturas de dados e algoritmos em Python*. Porto Alegre: Bookman, 2013.

SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms*. 4. ed. Boston: Addison-Wesley Professional, 2011.